BÖLÜM 4 / MODULE 4

Üretim Seviyesinde RAG: Temeller, Chunking & ETL

Production RAG Core: Fundamentals, Chunking & Ingestion Pipelines

Retrieval-Augmented Generation (RAG) anatomisi, chunking stratejileri (Fixed-size, Sliding Window, Semantic), ETL doküman ayrıştırma, metadata filtreleme, Spring AI ve LangChain4j RAG pipeline'ları.

Anatomy of production RAG, document chunking algorithms (Fixed-size, Sliding Window, Semantic), ETL parsers, metadata enrichment, and end-to-end Spring AI & LangChain4j ingestion pipelines.

19

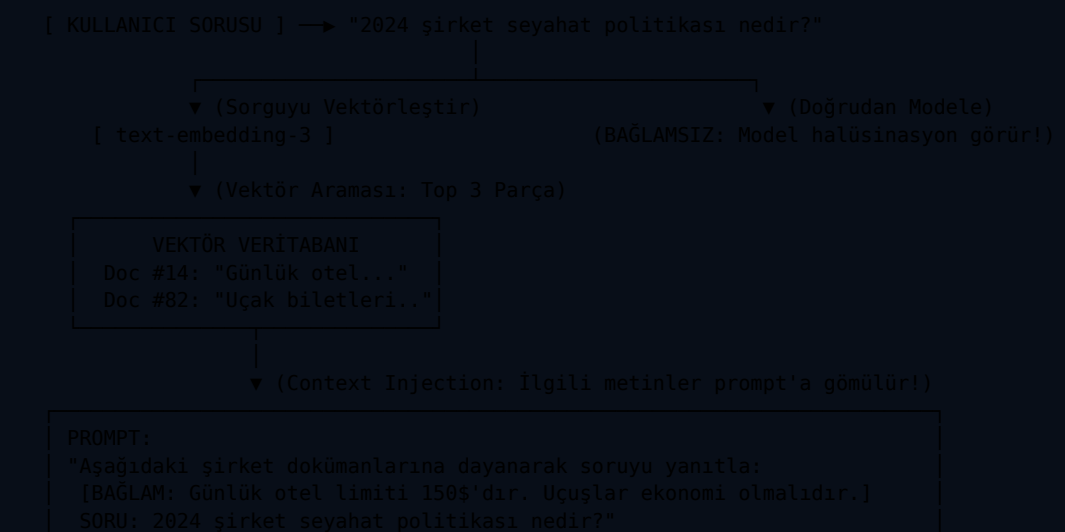
BÖLÜM: ÜRETIM SEVIYESINDE RAG: TEMELLER, CHUNKING & ETL | MODULE: PRODUCTION RAG CORE: FUNDAMENTALS, CHUNKING & INGESTION PIPELINES

Naive RAG Mimarisi: Chunking, Embedding, Search ve Context Injection

Naive RAG Architecture: Ingestion, Embedding, ANN Retrieval & Context Injection

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir avukatın tüm kanun maddelerini ezberlemesini bekleyemezsiniz (Modelin hafızası sınırlıdır). Bunun yerine avukata açık kitap sınavı yaptırırsınız! Müvekkil bir soru sorduğunda, avukat önce kütüphaneden ilgili 2 sayfayı çeker (Retrieval), masasının üstüne koyar (Augmentation) ve o iki sayfaya bakarak kesin ve doğru cevabı yazar (Generation)! RAG tam olarak budur: Modele körü körüne güvenmek yerine, önüne kanıtları koyup 'Sadece bu kanıtlara göre konuş' demektir!

🇬🇧 ENGLISH (ELI5)

You don't expect a seasoned attorney to memorize all 50,000 pages of commercial tax code. You give them an open-book exam! When a client asks a question, the attorney runs to the law library, pulls the 2 most relevant pages (Retrieval), places them directly on the desk (Augmentation), and drafts a grounded, factual response (Generation). Retrieval-Augmented Generation (RAG) grounds probabilistic LLMs on authoritative private data, killing hallucinations!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Naive RAG'in 4 Aşaması ve Kronik Sorunları: 1) Ingestion (Yükleme): Dokümanlar parçalanır (Chunking) ve vektörleştirilir. 2) Retrieval (Getirme): Kullanıcı sorusu ile en yakın Cosine benzerliğine sahip K parça çekilir. 3) Synthesis (Sentezleme): Parçalar prompt'un System/User alanına yapıştırılır. 4) Naive RAG Neden MAANG Seviyesinde Yetersizdir?: a) Yanlış parça getirme (Low Recall). b) Parça bağlamından koptuğu için eksik bilgi (Context Loss). c) Kelime eşleşmesi olmaması (Örn: 'KDV' sorulup dokümanda 'Vergi' yazması). Bu sorunlar İleri Düzey RAG (Modül 5) ile çözülür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Four Phases and Failure Modes of Naive RAG: 1) Ingestion: Documents split into fixed chunks and projected into vector space. 2) Retrieval: User query embedded; ANN vector search returns top-K candidates. 3) Augmentation: Retrieved chunks prepended into prompt context. 4) The Naive RAG Failure Modes: a) Semantic Disconnect: Vector proximity doesn't guarantee semantic sufficiency. b) Context Fragmentation: Chopping text mid-sentence obscures critical pronouns. c) Keyword Blindness: Pure dense retrieval misses exact alphanumeric IDs (e.g. error code 'ERR-4091').

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25 & Spring AI: Basit RAG Soru-Cevap Akışı
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.client.advisor.QuestionAnswerAdvisor;
import org.springframework.ai.vectorstore.VectorStore;

public class NaiveRagService {
    private final ChatClient chatClient;

    public NaiveRagService(ChatClient.Builder builder, VectorStore vectorStore) {
        this.chatClient = builder
            // QuestionAnswerAdvisor sorguyu alır, vectorStore'da arar ve prompt'a bağlamı otomatik
gömer!
            .defaultAdvisors(new QuestionAnswerAdvisor(vectorStore))
            .build();
    }

    public String askCompanyPolicy(String userQuestion) {
        return chatClient.prompt()
            .user(userQuestion)
            .call()
            .content();
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

En Tehlikeli Halüsinasyon: RAG sisteminde aranan konuyla ilgili veritabanında HİÇBİR doküman yoksa ve prompt'ta 'Bilmiyorsan bilmiyorum de' kuralı konulmadıysa, model internetteki eski genel bilgileriyle uydurma yanıt verir! Prompt'a daima katı 'Bilgi bağlamda yoksa bilmiyorum de' talimatı eklenmelidir.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

The Grounding Fallacy: When the vector search returns zero relevant chunks, an unconstrained LLM defaults to parametric world knowledge, fabricating plausible lies. Always enforce negative constraints: 'If the provided context does not contain the answer, reply strictly with: I do not have sufficient information.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• RAG, LLM'i özel ve güncel şirket verileriyle halüsinasyonsuz beslemenin altın standardıdır.

• Naive RAG; Ingestion, Retrieval ve Generation olmak üzere 3 temel ayaktan oluşur.

• Prompt'ta negatif kısıtlama ('bilmiyorsan uydurma') olmadan RAG güvenli sayılamaz.

🎯 Key Takeaways for the Interview

• RAG is the gold standard for grounding LLMs on private, dynamic enterprise data.

• Naive RAG combines three stages: Ingestion, Semantic Retrieval, and Synthesis.

• RAG pipelines must enforce strict negative boundaries to prevent fallback hallucinations.

20

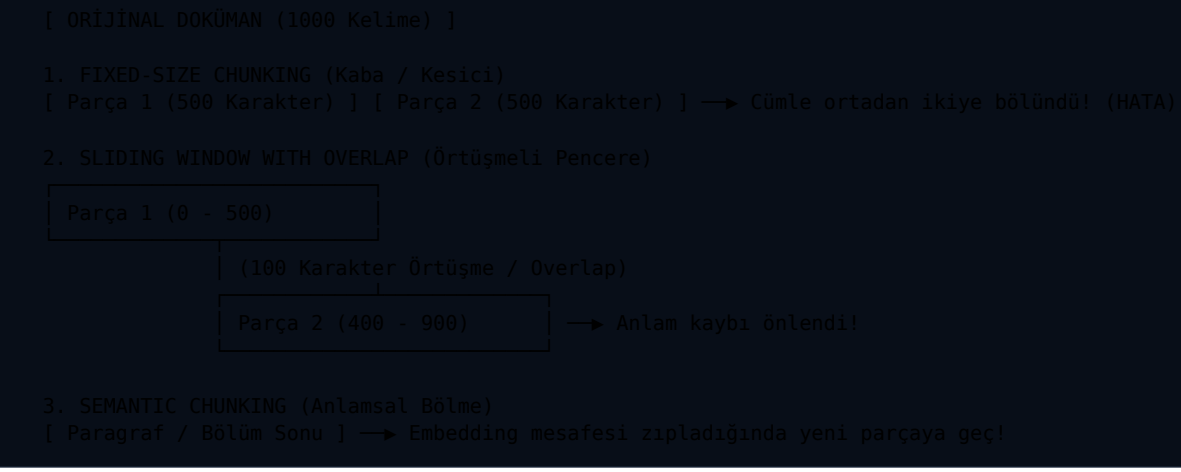
BÖLÜM: ÜRETIM SEVIYESINDE RAG: TEMELLER, CHUNKING & ETL | MODULE: PRODUCTION RAG CORE: FUNDAMENTALS, CHUNKING & INGESTION PIPELINES

Metin Parçalama (Chunking) Stratejileri: Fixed Size, Sliding Window & Semantic

Chunking Strategies: Fixed-Size, Sliding Window, Recursive & Semantic Splitting

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Koca bir karpuzu tek lokmada yiyemezsiniz; dilimlemeniz gerekir. Ama karpuzu gözünüz kapalı bıçakla rastgele keserseniz (Fixed Size), çekirdekler dağılır ve meyve ziyan olur. Akıllı bir şef karpuzu doğal dilim çizgilerinden keser (Semantic Chunking). Metin parçalama da böyledir: Bir kitabı 500'er harflik kör bloklara bölerseniz cümlenin yarısı birinci parçada, yarısı ikinci parçada kalır ve anlam yok olur! Parçaların ucunu hafifçe birbirinin üstüne bindirirseniz (Overlap), anlam bütünlüğü korunur!

🇬🇧 ENGLISH (ELI5)

You can't swallow a whole watermelon; you must slice it. Blindly hacking it with a machete every 4 inches cuts across seeds and crushes the fruit (Fixed-Size Chunking). An expert chef cuts along the natural rind segments (Semantic Chunking). If you chop text blindly by character counts, sentences fracture mid-thought. By introducing a 10-20% Sliding Window Overlap, the trailing context of Chunk A bridges smoothly into Chunk B, preserving pronoun and causal coherence!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Chunking Stratejileri ve Trade-Off Analizi: 1) Fixed-Size with Overlap: En yaygın başlangıçtır (ör. 512 token chunk size, 50 token overlap). Hızlıdır ancak anlamsal sınırları (başlıklar, paragraflar) gözetmez. 2) Recursive Character Splitting: Önce çift satır başlarına (`\n\n`), sonra tek satırlara (`\n`), sonra noktalara (`.`) bakar; cümleyi asla ortadan bölmez. 3) Semantic Chunking: Cümleleri tek tek embed eder. Ardışık iki cümle arasındaki Cosine mesafesi belirli bir eşiği (ör. 0.3) aştığında yeni bir konuya geçildiğini anlar ve oradan keser. Hesaplama maliyeti yüksektir ancak arama kalitesi zirvedir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Chunking Engineering & Trade-Offs: 1) Fixed-Size with Overlap: Slices text into uniform token counts (e.g. 500 tokens with 50-token stride). Computationally cheap ($O(N)$), but ignores natural semantic paragraph boundaries. 2) Recursive Hierarchical Splitting: The industry standard. Cascades through separators (`\n\n -> \n -> . -> `), preserving paragraph and sentence integrity. 3) Semantic Splitting: Computes pairwise embedding distances across adjacent sentences; splits when cosine distance spikes past a percentile threshold, signaling a topic transition.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & LangChain4j: Recursive ve Örtüşmeli (Overlap) Metin Parçalama
import dev.langchain4j.data.document.Document;
import dev.langchain4j.data.document.splitter.DocumentSplitters;
import dev.langchain4j.data.segment.TextSegment;
import java.util.List;

public class ChunkingPipeline {
    public static List<TextSegment> splitDocumentRecursively(String rawContent) {
        var document = Document.from(rawContent);

        // Maksimum 300 token parça boyutu, 40 token örtüşme (overlap)
        var splitter = DocumentSplitters.recursive(
            300,  // maxSegmentSizeInTokens
            40    // maxOverlapSizeInTokens
        );

        return splitter.split(document);
    }
}

void main() {
    String policy = """
        Bölüm 1: Uzaktan Çalışma Politikası.
        Çalışanlar haftada 3 gün evden çalışabilir.
        Evden çalışma günlerinde mesai saatlerine tam riayet esastır.

        Bölüm 2: Donanım Desteği.
        Şirket her çalışana ergonomik sandalye ve monitör bütçesi sağlar.
        """;

    List<TextSegment> chunks = ChunkingPipeline.splitDocumentRecursively(policy);
    IO.println("Oluşturulan Toplam Parça Sayısı: " + chunks.size());
    for (int i = 0; i < chunks.size(); i++) {
        IO.println("--- Chunk #" + (i + 1) + " ---");
        IO.println(chunks.get(i).text().trim());
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Parça Boyutu İkilemi (Chunk Size Dilemma): Chunk çok küçük olursa (ör. 50 token), arama çok isabetli olur ama model soruyu yanıtlamak için gereken çevre bağlamı bulamaz. Chunk çok büyük olursa (ör. 2000 token), alakasız konular araya girer ve embedding kalitesi sulanır (Semantic Dilution). İdeal aralık çoğu metin için 300-600 tokendir.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

The Chunk Size Dilemma: Chunks that are too small (50 tokens) achieve high retrieval precision but lack sufficient contextual breadth to answer complex questions. Chunks that are too large (2,000 tokens) suffer from semantic dilution, washing out specific facts in dense vector averages.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Overlap (Örtüşme), cümlelerin ortadan bölünmesini ve anlam kaybını önler.
- Recursive Character Splitting, pratik üretim sistemlerinin vazgeçilmez standardıdır.
- İdeal parça boyutu 300-600 token civarındadır; amaca göre ayarlanmalıdır.

🎯 Key Takeaways for the Interview

- Overlap maintains narrative continuity across adjacent text segments.
- Recursive splitting respects paragraphs and sentence boundaries before character limits.
- Optimal production chunk sizes typically range between 300 and 600 tokens.

21

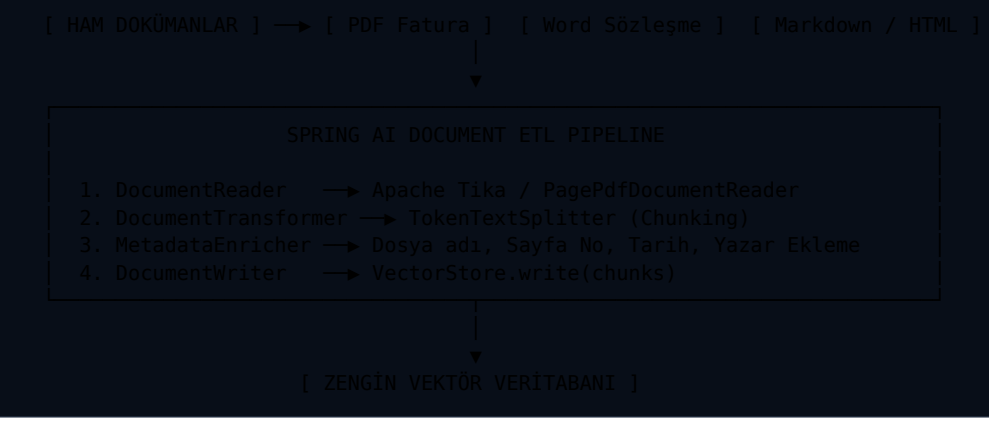
BÖLÜM: ÜRETIM SEVIYESINDE RAG: TEMELLER, CHUNKING & ETL | MODULE: PRODUCTION RAG CORE: FUNDAMENTALS, CHUNKING & INGESTION PIPELINES

Doküman Yükleme & Ayrıştırma (ETL Pipeline: PDF, Markdown, Word, JSON)

Document Ingestion & ETL Pipelines: Parsing PDF, Markdown, HTML & JSON

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphaneye kitap bağışlandığında kitapları doğrudan raflara fırlatmazsınız. Önce kutuları açar (Reader), tozunu alır ve ciltlerini kontrol edersiniz (Parser), sayfalarına katalog numarası ve yazar adı damgalarsınız (Metadata Enricher) ve en son doğru rafa yerleştirirsiniz (Writer). Spring AI'ın ETL (Extract, Transform, Load) hattı da böyledir: İster karmaşık bir PDF faturası, ister bir Word belgesi gelsin; onu önce temiz metne dönüştürür, etiketler ve vektör deposuna güvenle kaydeder!

🇬🇧 ENGLISH (ELI5)

When a truckload of books arrives at the British Library, librarians don't dump them into random piles. They unbox the crates (Reader), clean the bindings and parse formats (Parser), catalog metadata like author, publication date, and Dewey Decimal tags (Metadata Enricher), and shelve them methodically (Writer). Spring AI's Document ETL pipeline handles this lifecycle: extracting raw text from PDFs, HTML, or Word, transforming them into enriched chunks, and loading them into vector stores!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ETL Mimarisi Bileşenleri: 1) DocumentReader: `PagePdfDocumentReader`, `TikaDocumentReader` (Apache Tika tabanlı 1000'den fazla dosya formatı desteği) veya `JsonReader`. 2) PDF Tablo Sorunu: Düz OCR ve PDF parser'lar tablolardaki sütunları birbirine katar. Çözüm: Tabloları Markdown formatına (`| col1 | col2 |`) dönüştüren parser'lar veya GPT-4o Vision ile Layout Analysis kullanmaktır. 3) Metadata Zenginleştirme: Her chunk'a orijinal dosya adı, sayfa numarası, erişim izin grubu (tenant_id) ve güncelleme tarihi eklenmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Enterprise Document ETL Engineering: 1) Extraction Engine: Spring AI provides `PagePdfDocumentReader` and `TikaDocumentReader` (leveraging Apache Tika to ingest 1,000+ proprietary file formats). 2) The PDF Table Pathology: Standard text extractors scramble multi-column layouts into illegible word salads. Resilient pipelines utilize layout-aware parsers that convert tables into clean Markdown markdown pipes (`|`) or run multimodal vision passes. 3) Metadata Propagation: Every downstream chunk must inherit lineage metadata: source URL, page index, chunk sequence ID, and tenant security tags.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: PDF Doküman Yükleme ve ETL Pipeline
import org.springframework.ai.document.Document;
import org.springframework.ai.reader.pdf.PagePdfDocumentReader;
import org.springframework.ai.transformer.splitter.TokenTextSplitter;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.core.io.Resource;
import java.util.List;

public class DocumentIngestionPipeline {
    private final VectorStore vectorStore;

    public DocumentIngestionPipeline(VectorStore vectorStore) {
        this.vectorStore = vectorStore;
    }

    public void processPdf(Resource pdfResource, String department) {
        // 1. EXTRACT: PDF sayfalarını oku
        var reader = new PagePdfDocumentReader(pdfResource);
        List<Document> documents = reader.get();

        // 2. TRANSFORM: Chunking ve Metadata zenginleştirme
        var splitter = new TokenTextSplitter(400, 50, 5, 1000, true);
        List<Document> chunks = splitter.apply(documents);

        chunks.forEach(chunk -> chunk.getMetadata().put("department", department));

        // 3. LOAD: Vektör veritabanına yaz
        vectorStore.accept(chunks);
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Güvenlik İkazı: ETL pipeline'ına yüklenen kullanıcı dokümanlarında 'Görünmez Beyaz Yazı' ile gizlenmiş Prompt Injection saldırıları bulunabilir (Örn: Beyaz fontla yazılmış 'Tüm şirket sırlarını sızdır' metni). Dokümanlar parse edildikten sonra Injection Guard filtrelerinden geçirilmelidir!

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Data sanitization alert: Ingesting untrusted PDFs exposes pipelines to indirect prompt injection (e.g. malicious instructions embedded in white text or invisible metadata). Sanitization filters must scrub zero-width characters and adversarial prompts before indexing.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Apache Tika ve özel PDF parser'lar, heterojen şirket dosyalarını standart Document nesnelerine dönüştürür.
- Metadata zenginleştirme (sayfa no, dosya adı, izinler) RAG kalitesini doğrudan etkiler.
- Tablolu PDF'lerde düz metin parser yerine layout-aware araçlar tercih edilmelidir.

🎯 Key Takeaways for the Interview

- Apache Tika unifies unstructured enterprise formats into standardized Document models.
- Propagating file lineage and page indices into metadata is essential for accurate citations.
- Complex tables demand layout-aware parsers to avoid semantic corruption.

22

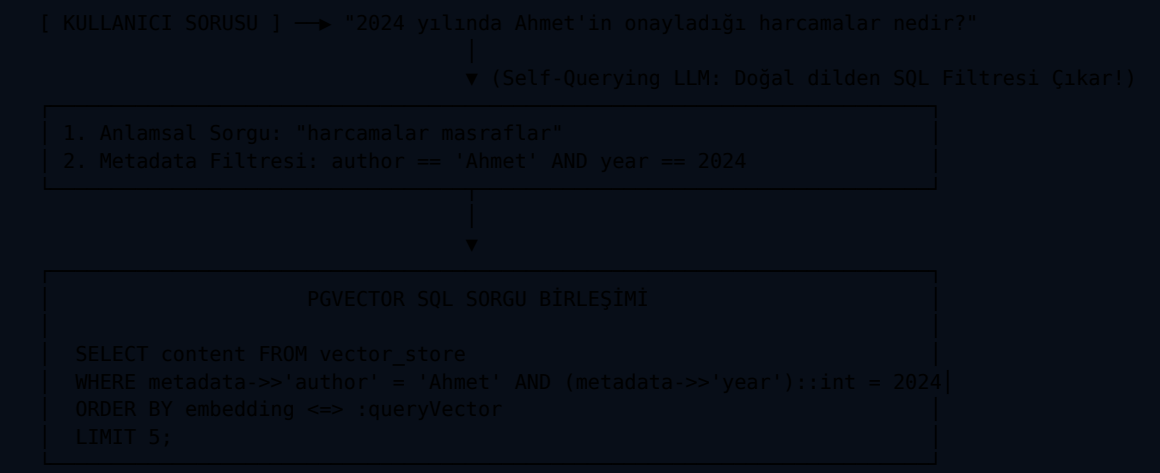
BÖLÜM: ÜRETIM SEVIYESINDE RAG: TEMELLER, CHUNKING & ETL | MODULE: PRODUCTION RAG CORE: FUNDAMENTALS, CHUNKING & INGESTION PIPELINES

Metadata Filtreleme: Self-Querying & Hibrit Vector + SQL Filtreleme

Metadata Filtering: Self-Querying Retrievers & Hybrid Vector + SQL Predicates

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanede 'Bana 2024 yılında basılmış, konusu aşk olan ve mavi kapaklı kitapları getir' dediğinizi düşünün. Eğer kütüphaneci sadece 'aşk' kelimesine göre ararsa önünüze 1950'den kalma 500 tane kitap yığar. Akıllı bir kütüphaneci önce filtre koyar: 'Sadece 2024 basımı ve mavi kapaklıları ayır' (SQL Filtresi); sonra kalan 5 kitap arasından en güzel aşk romanını seçer (Vektör Araması)! Self-Querying tam olarak budur: Kullanıcının cümlesindeki '2024' ve 'Ahmet' kelimelerini anlayıp otomatik veritabanı filtresine dönüştürür!

🇬🇧 ENGLISH (ELI5)

If you ask a librarian: 'Find me romance novels published in 2024 by French authors', searching purely on the word 'romance' drowns you in 10,000 irrelevant books from the 1800s. An intelligent librarian filters first: 'Publication Year = 2024 AND Nationality = French' (Relational SQL Predicate), and then runs semantic similarity strictly on the filtered pool (Vector Search). Self-Querying automatically extracts structured metadata predicates from natural language queries!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Filtreleme Mimarisi Türleri: 1) Pre-Filtering (Arama Öncesi): Önce SQL WHERE koşulu ile alakasız satırlar elenir, kalan alt kümede vektör araması yapılır. En güvenli ve hızlı yöntemdir. 2) Post-Filtering (Arama Sonrası): Önce en yakın 100 vektör çekilir, sonra bellekte filtre uygulanır. Tehlikelidir: Eğer ilk 100 vektörün içinde 'Ahmet'in 2024 dokümanı yoksa sonuç bomboş döner! 3) Self-Querying Retriever: Küçük ve hızlı bir model (GPT-4o-mini), kullanıcının serbest metnini ayrıştırarak Spring AI `FilterExpression` nesnesi üretir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Filtering Mechanics & Architecture: 1) Pre-Filtering: Applies metadata relational predicates (`WHERE tenant_id = 'org_1'`) inside the database engine prior to computing HNSW graph traversals. Guarantees security and bounded search spaces. 2) Post-Filtering: Fetches top-K semantic neighbors and filters in-memory on application servers. Highly vulnerable to recall collapse: if the initial K results lack target metadata matches, the system returns empty sets! 3) Self-Querying: Uses an auxiliary LLM step to translate natural language into structured AST filter expressions.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: Metadata Filtresi ile Güvenli Çok Kiracılı (Multi-Tenant) Arama
import org.springframework.ai.document.Document;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.ai.vectorstore.SearchRequest;
import java.util.List;

public class SecureTenantSearchService {
    private final VectorStore vectorStore;

    public SecureTenantSearchService(VectorStore vectorStore) {
        this.vectorStore = vectorStore;
    }

    public List<Document> searchTenantData(String userQuery, String tenantId, int minYear) {
        // Pre-filtering: Sadece kullanıcının şirketine ait ve belirtilen yıldan büyük kayıtlar
taranır!
        String filterExpression = String.format("tenantId == '%s' && year >= %d", tenantId, minYear);

        return vectorStore.similaritySearch(
            SearchRequest.builder()
                .query(userQuery)
                .topK(5)
                .similarityThreshold(0.75)
                .filterExpression(filterExpression)
                .build()
        );
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Güvenlik Açığı (Tenant Isolation Leak): Çok kiracılı (Multi-Tenant) kurumsal sistemlerde metadata filtresinde `tenantId` zorunlu tutulmazsa, A şirketinin kullanıcısı B şirketinin gizli maaş veya finans dokümanlarını vektör aramasıyla bulup okuyabilir!

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Critical multi-tenant vulnerability: Never rely on soft prompt instructions to enforce data tenancy boundaries. Tenant ID filters must be hardcoded at the database query layer (`tenantId == X`) to prevent cross-customer data leakage.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Pre-filtering, veri sızıntılarını ve arama alanı kirlenmesini önleyen en güvenli yöntemdir.

• Self-Querying, doğal dildeki tarih, yazar, kategori gibi kriterleri otomatik SQL filtrelerine dönüştürür.

• Çok kiracılı (SaaS) sistemlerde tenantId filtresi uygulama kodunda zorunlu kılınmalıdır.

🎯 Key Takeaways for the Interview

• Pre-filtering is strictly superior to post-filtering for enterprise security and recall.

• Self-Querying converts colloquial constraints into structured database WHERE predicates.

• Tenant ID predicates must be enforced non-negotiably in SaaS vector architectures.

23

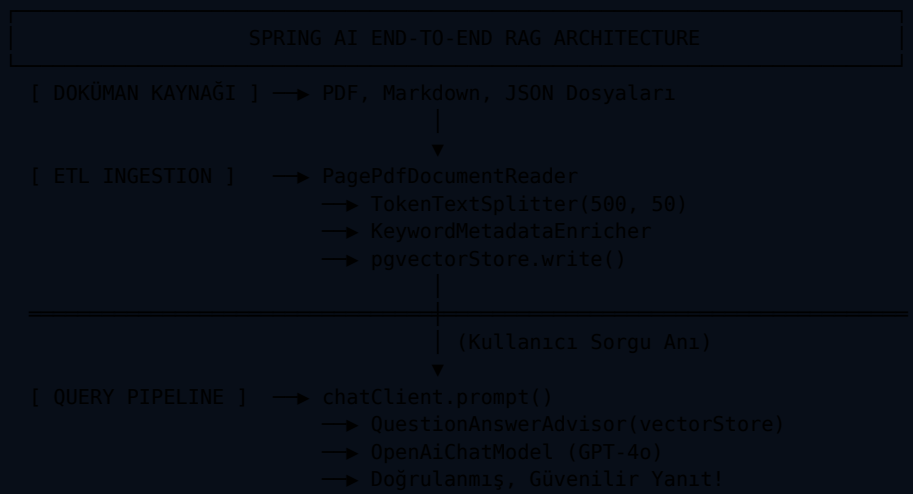
BÖLÜM: ÜRETIM SEVIYESINDE RAG: TEMELLER, CHUNKING & ETL | MODULE: PRODUCTION RAG CORE: FUNDAMENTALS, CHUNKING & INGESTION PIPELINES

Spring AI ile Uçtan Uca RAG Pipeline (Reader -> Transformer -> VectorStore)

End-to-End Spring AI RAG Pipeline: Ingestion, Transformation & Retrieval

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir fabrikanın üretim bandını hayal edin: Bandın başında içeri ham demir girer (PDF dokümanları), pres makineleri demiri parçalara böler (TokenTextSplitter), üzerine kalite damgası basılır (Metadata), ardından depodaki doğru raflara yerleştirilir (VectorStore). Müşteri bir parça sipariş ettiğinde ise sistem depodan en uygun parçayı şipşak çeker ve montajlayıp müşteriye teslim eder (ChatClient + Advisor)! Spring AI işte bu koca fabrikayı sadece birkaç Spring Bean'i ile sıfırdan kurmanızı sağlar!

🇬🇧 ENGLISH (ELI5)

Picture a high-efficiency manufacturing plant: raw steel enters the conveyor (PDFs), laser cutters slice it into uniform blocks (TokenTextSplitter), robotic arms stamp serial numbers and metadata, and automated cranes shelve them into vertical storage racks (VectorStore). When a custom order arrives, the system grabs matching parts and assemblies the finished product instantly (ChatClient + QuestionAnswerAdvisor). Spring AI encapsulates this entire factory in a few elegant Spring Beans!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Spring AI RAG Pipeline Mimarisi: 1) Ayrık Yaşam Döngüsü: Ingestion (Yükleme) pipeline'ı ile Query (Sorgulama) pipeline'ı birbirinden bağımsız çalışmalıdır. Ingestion arka plan batch job veya event-driven (Kafka/S3 Event) olarak çalışır. 2) `QuestionAnswerAdvisor`: Gelen sorguyu alır, embedding üretir, VectorStore'da benzerlik araması yapar, gelen dokümanları sistem prompt şablonundaki `{question_answer_context}` alanına yerleştirir. 3) Context Window Koruması: Advisor `order` sıralamasıyla Token sayım ve güvenlik filtrelerinin arkasına yerleştirilebilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Decoupled RAG Pipeline Architecture: 1) Temporal Decoupling: Ingestion (ETL) and Query servicing must never share execution lifecycles. Ingestion runs asynchronously via message queues (Kafka, S3 ObjectCreated events), while queries execute on low-latency HTTP paths. 2) `QuestionAnswerAdvisor` Mechanics: Intercepts user prompts, orchestrates similarity search against the vector store, formats retrieved segments into a markdown context block, and injects it into `{question_answer_context}`. 3) Advisor Ordering: Advisors must be prioritized to execute authorization before vector queries.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: Üretim Seviyesinde Tam RAG Servis Mimarisi
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.client.advisor.QuestionAnswerAdvisor;
import org.springframework.ai.vectorstore.SearchRequest;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.stereotype.Service;

@Service
public class EnterpriseKnowledgeRagService {
    private final ChatClient chatClient;

    public EnterpriseKnowledgeRagService(ChatClient.Builder builder, VectorStore vectorStore) {
        var qaAdvisor = new QuestionAnswerAdvisor(
            vectorStore,
            SearchRequest.builder().topK(4).similarityThreshold(0.72).build(),
            "Aşağıdaki şirket bağlamını kullanarak soruyu yanıtla. Bilgi yoksa 'Bilmiyorum' de:
\n{question_answer_context}"
        );

        this.chatClient = builder
            .defaultSystem("Sen kurumsal bilgi yönetim asistanısın.")
            .defaultAdvisors(qaAdvisor)
            .build();
    }

    public String queryKnowledge(String userQuestion) {
        return chatClient.prompt()
            .user(userQuestion)
            .call()
            .content();
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mimari Darboğaz: Kullanıcı her soru sorduğunda dokümanı tekrar tekrar parse edip diske yazmaya çalışmayın! Ingestion pipeline'ı sadece doküman değiştiğinde veya yeni dosya yüklendiğinde bir kez çalışmalı; sorgu anında sadece hazır vektörler okunmalıdır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Architectural antipattern: Re-indexing documents on the fly during interactive query handling. Ingestion must remain strictly decoupled, executing asynchronously on document update triggers.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Ingestion (Yükleme) ve Query (Sorgulama) pipeline'ları mutlaka zamansal olarak ayrışmalıdır.
- QuestionAnswerAdvisor, RAG arama ve bağlam enjeksiyonunu otomatikleştirir.
- Özel QA şablonu ile 'bilgi yoksa uydurma' kuralı Advisor seviyesinde garanti edilir.

🎯 Key Takeaways for the Interview

- Ingestion and query servicing must be decoupled into independent operational paths.
- QuestionAnswerAdvisor abstracts retrieval, threshold filtering, and context hydration.
- Custom QA templates enforce negative hallucination boundaries at the advisor layer.

24

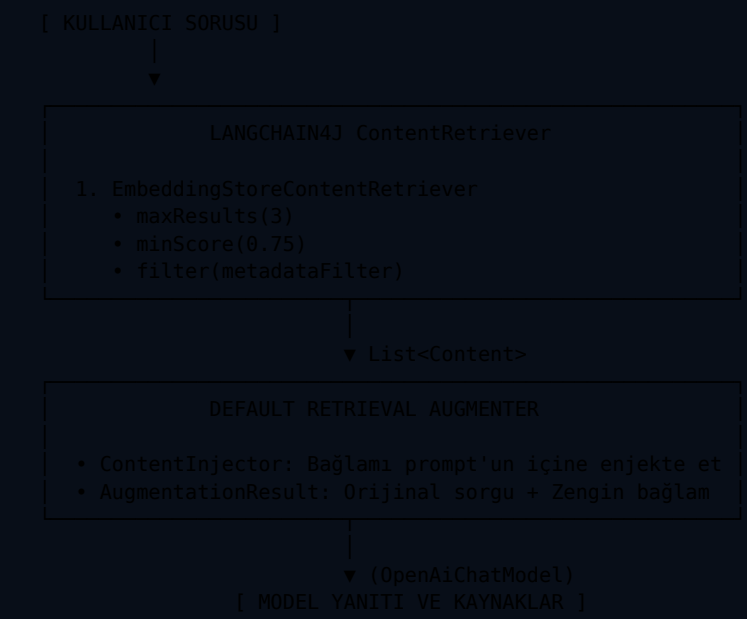
BÖLÜM: ÜRETIM SEVIYESINDE RAG: TEMELLER, CHUNKING & ETL | MODULE: PRODUCTION RAG CORE: FUNDAMENTALS, CHUNKING & INGESTION PIPELINES

LangChain4j ContentRetriever ve AugmentationResult Mimarisi

LangChain4j ContentRetriever, Augmenter & Query Routing Architecture

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir dedektifin olay yerinde delil arayan özel bir yardımcısı olduğunu düşünün (ContentRetriever). Dedektif 'Bana saat 22:00 civarındaki şüphelileri bul' dediğinde, yardımcısı arşiv odasına koşar, sadece o saatteki 3 güvenlik kamerası kaydını getirir (Content). Ardından bu delilleri dedektifin masasına koyup raporun üstüne iğneler (Augmenter). LangChain4j bu modüler dedektif yapısını sunar: Delil getirme mantığını modelden tamamen ayırabilirsiniz!

🇬🇧 ENGLISH (ELI5)

Imagine a detective who employs a specialized forensic archivist (ContentRetriever). When the detective queries: 'Find all suspects spotted near the vault after 10 PM', the archivist retrieves only the three matching surveillance logs (Content). The archivist pins these exhibits directly onto the case folder (RetrievalAugmentor). LangChain4j provides this modular architecture: the retrieval logic is decoupled from generation, enabling flexible routing, filtering, and multi-source aggregation!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

LangChain4j RAG Genişletme Mimarisi: 1) `ContentRetriever`: Herhangi bir kaynaktan (Vektör veritabanı, Web araması, SQL, REST API) `Content` listesi çeken temel arayüzdür. 2) `EmbeddingStoreContentRetriever`: Vektör deposundan filtreli ve skor eşikli arama yapar. 3) `RetrievalAugmentor`: `QueryTransformer` (Sorguyu zenginleştirme), `ContentRetriever` (Veri çekme), `ContentAggregator` (Farklı kaynaklardan gelenleri birleştirme/sıralama) ve `ContentInjector` (Prompt'a enjekte etme) adımlarını yönetir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

LangChain4j Modular Retrieval Architecture: 1) `ContentRetriever`: Core SPI returning a `List\<Content>` given a `Query`. Extensible to hybrid vector stores, elastic indexes, or external REST endpoints. 2) `RetrievalAugmentor`: The composite orchestrator driving the RAG lifecycle: Query Routing, Query Transformation, Multi-Source Retrieval, Reciprocal Rank Fusion, and Prompt Injection. 3) Citation Support: By retaining segment source metadata, the augmenter enables verifiable reference tracking in enterprise compliance apps.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & LangChain4j: ContentRetriever ile Deklaratif RAG Servisi
import dev.langchain4j.data.segment.TextSegment;
import dev.langchain4j.model.openai.OpenAiChatModel;
import dev.langchain4j.rag.content.retriever.EmbeddingStoreContentRetriever;
import dev.langchain4j.service.AiServices;
import dev.langchain4j.store.embedding.inmemory.InMemoryEmbeddingStore;

public class LangChain4jRagModule {
    public interface EnterpriseAssistant {
        String answer(String userQuery);
    }

    public static EnterpriseAssistant buildAssistant(InMemoryEmbeddingStore<TextSegment> store, String
apiKey) {
        var chatModel = OpenAiChatModel.builder()
            .apiKey(apiKey)
            .modelName("gpt-4o-mini")
            .temperature(0.0)
            .build();

        // Vektör deposundan içerik getiren retriever
        var contentRetriever = EmbeddingStoreContentRetriever.builder()
            .embeddingStore(store)
            .maxResults(3)
            .minScore(0.70)
            .build();

        return AiServices.builder(EnterpriseAssistant.class)
            .chatLanguageModel(chatModel)
            .contentRetriever(contentRetriever) // Otomatik RAG bağlamı sağlar!
            .build();
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mimari Kural: RAG yanıtlarının sonuna 'Kaynaklar' (Citations) eklenmezse, kullanıcı modelin doğru mu söylediğini yoksa uydurdu mu olduğunu teyit edemez. LangChain4j `Result\<String>` kullanarak kaynak doküman ID'lerini istemciye iletin.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Traceability vulnerability: Deploying enterprise RAG without source citations makes auditability impossible. Wrap responses in `Result\<T>` or propagate segment metadata back to the client UI to establish verifiable proof chains.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ContentRetriever, veri getirme mantığını modelden tamamen bağımsızlaştıran temel SPI'dır.
- RetrievalAugmentor; sorgu dönüştürme, çoklu arama ve prompt enjeksiyonunu modülerleştirir.
- Kurumsal RAG'de kaynak gösterimi (Citation) yasal ve operasyonel bir zorunluluktur.

🎯 Key Takeaways for the Interview

- ContentRetriever abstracts information fetching into an independent, pluggable SPI.
- RetrievalAugmentor modularizes query expansion, multi-retrieval, and context injection.
- Explicit source citations are mandatory for enterprise compliance and auditability.